Claude Prompt - Build Enterprise SolarWinds AI Operations Agent for TechAdmin

I am providing:

1. **TechAdmin source code ZIP**
2. **SolarWinds Jupyter Notebook (.ipynb)** containing working SolarWinds SWIS integration
3. Existing database models and migrations
4. Existing TechAdmin Agent architecture
5. Existing ticket generation code and workflows

Your task is to thoroughly analyze the entire codebase and implement a new production-grade **SolarWinds AI Operations Agent** that follows all existing TechAdmin architectural patterns, coding standards, authentication mechanisms, logging practices, database conventions, UI design principles, and AI agent framework implementations.

Do not create a standalone prototype.

The solution must be integrated into TechAdmin as a fully supported enterprise feature.

Business Objective

Create an AI-powered SolarWinds Operations Agent capable of:

- Automated SolarWinds telemetry ingestion
- Historical data storage
- Device health scoring
- Trend analysis
- Forecasting
- AI-generated operational reports
- Automated incident identification
- Automated ticket generation
- Ticket lifecycle visibility
- Natural language querying through an AI Agent

- Dashboard visualization
- Executive reporting

This will become the third major operational agent within TechAdmin.

Existing SolarWinds Integration

Analyze the notebook and existing SolarWinds client implementation.

Leverage existing code where possible.

Current telemetry retrieval already provides:

NodeID
Caption
IPAddress
Status
StatusDescription
CPULoad
PercentMemoryUsed
ResponseTime
AvgResponseTime
Vendor
MachineType
LastSync
SystemUpTime
NodeDescription
City
ITSM_Location
NodeCategory
NodeType
Contact

The new implementation should reuse proven extraction logic wherever appropriate.

Agent Overview

New Agent Name:

SolarWinds AI Operations Agent

Purpose:

Monitor
Analyze
Forecast
Recommend
Create Tickets
Provide AI Reports

Required Folder Structure

Follow existing TechAdmin design patterns.

Recommended structure:

```
App
 │
 ├──── services
 │     └──── solarwinds
 │           ├──── service.py
 │           ├──── repository.py
 │           ├──── telemetry_collector.py
 │           ├──── health_engine.py
 │           ├──── forecasting_engine.py
 │           ├──── ai_reporting_engine.py
 │           ├──── ticket_engine.py
 │           ├──── scheduler.py
 │           └──── prompts.py
 │
 ├──── agents
```

```
│   └── solarwinds_agent
│       ├── graph.py
│       ├── nodes.py
│       ├── tools.py
│       ├── prompts.py
│       ├── state.py
│       └── models.py
│
├── database
│   ├── models
│   └── migrations
│
└── api
```

Adapt to existing architecture discovered in the ZIP.

Database Requirements

Design proper relational schema.

Do not overwrite historical data.

Store historical snapshots.

Table 1: Device Master

solarwinds_devices

Purpose:

Master inventory
One record per device

Columns:

id BIGSERIAL PK

node_id BIGINT UNIQUE
caption VARCHAR(500)
ip_address VARCHAR(200)
vendor VARCHAR(200)
machine_type VARCHAR(500)
node_category VARCHAR(100)
node_type VARCHAR(100)
city VARCHAR(200)
itsm_location VARCHAR(500)
contact VARCHAR(500)
current_status INTEGER
current_status_description VARCHAR(100)
created_at TIMESTAMP
updated_at TIMESTAMP

Table 2: Daily Telemetry Snapshots

solarwinds_device_snapshots

Purpose:

Historical telemetry
One snapshot per collection cycle

Columns:

snapshot_id BIGSERIAL PK
snapshot_date DATE
snapshot_timestamp TIMESTAMP
node_id BIGINT
cpu_load DECIMAL(10,2)
memory_used_pct DECIMAL(10,2)
response_time_ms DECIMAL(10,2)
avg_response_time_ms DECIMAL(10,2)
status INTEGER
status_description VARCHAR(100)
health_current VARCHAR(20)

system_uptime BIGINT
last_sync TIMESTAMP
created_at TIMESTAMP

Table 3: AI Insights

solarwinds_ai_insights

Columns:

insight_id BIGSERIAL PK
node_id BIGINT
generated_at TIMESTAMP
risk_level VARCHAR(50)
risk_score DECIMAL
forecast_summary TEXT
root_cause TEXT
recommendation TEXT
llm_analysis TEXT
ticket_generated BOOLEAN

Table 4: Forecast Results

solarwinds_forecasts

Columns:

forecast_id BIGSERIAL PK
node_id BIGINT
forecast_date DATE
predicted_cpu DECIMAL
predicted_memory DECIMAL
predicted_health VARCHAR(50)
predicted_risk VARCHAR(50)
forecast_confidence DECIMAL
generated_at TIMESTAMP

Table 5: Ticket Mapping

solarwinds_generated_tickets

Columns:

id BIGSERIAL PK
node_id BIGINT
ticket_id VARCHAR(200)
ticket_status VARCHAR(100)
ticket_reason TEXT
created_at TIMESTAMP
updated_at TIMESTAMP

Automated Ingestion Framework

Implement automatic telemetry collection.

Daily Collection

Daily:

08:00 AM

Workflow:

Connect SolarWinds
Fetch telemetry
Validate data
Calculate health
Store snapshot
Generate insights
Check forecast
Generate tickets if required

Create AI summary

Scheduling

Analyze existing scheduler design in TechAdmin.

Support:

Windows Task Scheduler
APScheduler
Cron
Future Celery support

Provide recommendation and implementation.

Health Calculation Engine

Implement centralized health scoring service.

LOW

If any condition is true:

Status = Down
Status = Critical
Status = Unreachable

OR

CPU > 90%

OR

Memory > 90%

Then:

HealthCurrent = Low

MEDIUM

If not Low and:

CPU >= 75%

OR

Memory >= 80%

OR

Status:

Warning
Shutdown
Unmanaged
Unknown
Inactive
Undefined

Then:

HealthCurrent = Medium

HIGH

If:

Status = Up

AND

CPU < 75%

AND

Memory < 80%

Then:

HealthCurrent = High

Important Requirement

Missing CPU and Memory values must NOT downgrade healthy devices.

Example:

Status = Up
CPU = NULL
RAM = NULL
HealthCurrent = High

Forecasting Engine

Implement historical forecasting.

Use snapshot history from database.

Provide:

7 day forecast
30 day forecast
90 day forecast

Forecast:

CPU Growth
Memory Growth
Health Degradation
Response Time Trend

Generate:

Forecast Risk Score

Forecast Health Status

Support future replacement with ML models.

Design forecasting abstraction layer.

AI Reporting Engine

Generate AI reports.

The agent should produce:

Daily Executive Report

Example:

1590 monitored devices.
1320 High Health
190 Medium Health
80 Low Health
Network category contains the largest concentration of at-risk assets.
Average fleet CPU utilization is 42%.
Average fleet memory utilization is 68%.
12 devices entered Low Health today.

Risk Report

Example:

Server XYZ exceeded CPU threshold.
Memory utilization exceeded 90%.
Multiple response time spikes detected.
Risk of service degradation is high.

Operational Recommendation Report

Example:

Immediate investigation recommended.
Review running services.
Evaluate memory leaks.
Check patch status.
Review recent configuration changes.

AI-Powered Questions

The SolarWinds Agent should answer:

Current State

Show all unhealthy devices.
Show all low health devices.
How many devices are critical?
Show network devices with high CPU.

Trends

Which devices degraded in last 30 days?
Show health trend for device X.
Which categories are most unstable?

Forecasts

Which devices are likely to fail next week?
Which servers will exceed 90% CPU?
Predict memory exhaustion risks.

Tickets

Show generated tickets.
Why was this ticket created?
Which low health devices do not have tickets?

Executive

Generate daily report.
Generate weekly report.
Summarize operational risks.
Top 20 devices requiring attention.

Automated Ticket Generation

Analyze existing ticketing framework inside TechAdmin.

Reuse available ticket service.

Do not create duplicate ticket functionality.

Ticket Creation Conditions

Create incident automatically when:

HealthCurrent = Low

OR

Forecast indicates:

Risk of entering Low Health

within configured threshold period.

Ticket Payload

Must include:

Node Name
Node ID
IP Address
Category
Current Status
CPU
Memory
HealthCurrent
Risk Score
Forecast
Root Cause
Recommendation
Generated Timestamp

Duplicate Prevention

Implement logic to avoid:

Multiple open tickets for same issue

Use:

Node ID
Issue Type
Health State
Time Window

for deduplication.

Dashboard Enhancements

Create dedicated SolarWinds dashboard.

Summary Cards

Total Devices
High Health
Medium Health
Low Health
Open Tickets
Predicted Risks

Category Distribution

Network
Server
Telecom
Cloud
Collaboration
URL
Client IP

Vendor Distribution

Cisco
Windows
Linux
SolarWinds
Other

Trend Graphs

CPU Trend
Memory Trend
Response Time Trend
Device Growth Trend
Health Trend

Forecast Widgets

Forecasted Failures
Forecasted Capacity Issues
Predicted High Risk Devices

Security

Re-use existing TechAdmin guardrails.

Ensure:

Read-only SolarWinds access
No write operations to SolarWinds
RBAC compliance
Audit Logging
Error Tracking
Input Validation
Secure Secrets Management

Performance Requirements

Must scale to:

1,500 – 10,000+ devices

Optimize:

Pagination
Batch Inserts
Database Indexing
Caching
Asynchronous Processing

Testing Requirements

Generate:

Unit Tests
Integration Tests
Forecasting Tests
Ticket Creation Tests
Database Tests
Agent Tool Tests
Dashboard Tests

Deliverables

Provide:

1. Complete gap analysis of current TechAdmin architecture.
2. Recommended implementation architecture.
3. Database schema.
4. Alembic/database migration scripts.
5. New service layer implementation.
6. Repository layer implementation.
7. Scheduler implementation.
8. Forecasting engine implementation.
9. AI reporting engine.
10. Ticket integration.
11. Agent integration.

12. Dashboard integration.
13. API routes.
14. Test strategy.
15. Deployment strategy.
16. File-by-file explanation of every created/modified component.
17. Future roadmap for real-time monitoring and advanced predictive analytics.

Before coding, fully analyze the supplied ZIP and notebook, identify reusable components, and implement the solution in a way that minimizes duplication and aligns with the existing TechAdmin architecture and coding standards.